

MODULE 21

LSTM & GRU — In-Depth Intuition

Lectures Covered

- Lecture 1 — From RNN to LSTM
- Lecture 2 — Different Memory Types in LSTM
- Lecture 3 — Gates in LSTM
- Lecture 4 — GRU Introduction
- Lecture 5 — Working of GRU

Lecture 1: From RNN to LSTM

Quick Recap — RNN Hidden State

A vanilla RNN maintains a single hidden state updated at each time step:

```
# h_t = tanh( W_h * h_(t-1) + W_x * x_t + b )

# h_t = hidden state (also the memory)
# x_t = input at time t
# W_h, W_x = learnable weight matrices
```

The Vanishing Gradient Problem

During BPTT, gradients are multiplied by the weight matrix at every time step. If weights are slightly < 1 , the gradient shrinks exponentially — the network forgets early inputs.

```
# Sequence of 100 words:
# Gradient ~ 0.9^100 ~ 0.000027 -> effectively zero

# The model cannot learn that 'cat' (word 1) determines
# verb agreement 80 words later.
```

Vanishing vs Exploding Gradients

Vanishing: weights < 1 , gradients $\rightarrow 0$ $\rightarrow$ early layers receive no update. Hard to fix.
Exploding: weights > 1 , gradients $\rightarrow \text{inf}$ $\rightarrow$ NaN loss. Fixed by gradient clipping.
LSTM addresses vanishing; gradient clipping handles exploding.

LSTM — The Solution

LSTM (Hochreiter & Schmidhuber, 1997) introduces a separate Cell State C_t as a long-range memory highway alongside the hidden state h_t .

Aspect	RNN vs LSTM
Memory	Single h_t vs Cell state C_t + hidden state h_t
Long-term memory	Poor vs Good (additive cell update)
Vanishing gradient	Severe vs Largely mitigated
Gates	None vs 3 gates (Forget, Input, Output)
Best use case	Short sequences vs Long sequences, NLP, speech

Core intuition

h_t = short-term memory (what we are thinking right now).
 C_t = long-term memory (important facts remembered from much earlier).
Like reading a book: working memory holds the current page,
but you still remember the villain introduced in Chapter 1.

Lecture 2: Different Memory Types in LSTM

Two Memory Streams

	Cell State (C_t)	Hidden State (h_t)
Also called	Long-term memory	Short-term / Working memory

	Cell State (C _t)	Hidden State (h _t)
Role	Carries info across long sequences	Output at each step; short context
Modification	Subtle adds/removals via gates	Recomputed every step
Analogy	Notebook of important facts	What you are currently thinking

Cell State — The Memory Highway

C_t runs the full length of the sequence with minimal modification. Information travels through it unless the gates decide to change it.

- Forgetting: old info selectively erased by the Forget Gate.
- Writing: new info added by the Input Gate + Candidate values.

Why gradients flow better through C_t

Cell update is additive: $C_t = f_t * C_{(t-1)} + i_t * C_{\sim t}$

Addition means gradients do not decay — they flow back largely unchanged.

This is why LSTM learns long-range dependencies that plain RNNs cannot.

Hidden State — Working Memory

The hidden state h_t is derived from the cell state and serves as the output at each step:

```
# h_t = o_t * tanh( C_t )

# o_t = output gate (0 to 1) — how much of C_t to expose
# h_t is a filtered, compressed view of long-term memory
```

Editor analogy

LSTM is a skilled editor reviewing text step by step.

At each step: look at the notebook C_(t-1), current input x_t, last thought h_(t-1).

Decide what to erase, what to add, what to output — then pass both C_t and h_t forward.

Lecture 3: Gates in LSTM

What Are Gates?

Each gate is a sigmoid layer producing values in (0, 1) — acting as a soft switch controlling information flow.

Gate value	Meaning
~0	Block everything — forget / let nothing through
~1	Allow everything — keep fully / pass all through
In between	Partial memory retention or partial output

Gate 1: Forget Gate (f_t)

Decides what fraction of the old cell state C_(t-1) to discard:

```
# f_t = sigmoid( W_f * [h_(t-1), x_t] + b_f )

# f_t element-wise multiplied with C_(t-1)
# f_t[i] ~ 0 -> that memory slot is erased
# f_t[i] ~ 1 -> that memory slot is fully retained
```

Example

Starting a new paragraph: the subject gender from the previous sentence is irrelevant.
Forget gate fires high for that slot -> cleared for the new subject.

Gate 2: Input Gate (i_t) + Candidate (C_t)

Two-part mechanism to write new information into the cell state:

```
# Part A - Input Gate: what positions to update
# i_t = sigmoid( W_i * [h_(t-1), x_t] + b_i )

# Part B - Candidate: what values to potentially write
# C_t = tanh( W_C * [h_(t-1), x_t] + b_C )

# Combined cell state update:
# C_t = f_t * C_(t-1) + i_t * C_t
#      ^--- forget old   ^--- write new
```

Example

Reading a new subject name: input gate fires, candidate encodes
'new subject is plural' -> written into cell state for future agreement.

Gate 3: Output Gate (o_t)

Decides what part of the cell state to expose as the hidden state h_t :

```
# o_t = sigmoid( W_o * [h_(t-1), x_t] + b_o )
# h_t = o_t * tanh( C_t )

# tanh(C_t) rescales cell state to [-1, 1]
# o_t filters how much gets passed as output / next h_t
```

Full LSTM Step — Summary

Gate	Formula	Role
Forget Gate	$f_t = \text{sigmoid}(W_f[h_{(t-1)}, x_t] + b_f)$	Erase irrelevant long-term memory
Input Gate	$i_t = \text{sigmoid}(W_i[h_{(t-1)}, x_t] + b_i)$	How much new info to write
Candidate	$C_t = \tanh(W_C[h_{(t-1)}, x_t] + b_C)$	New values to potentially write
Cell Update	$C_t = f_t * C_{(t-1)} + i_t * C_t$	Updated long-term memory
Output Gate	$o_t = \text{sigmoid}(W_o[h_{(t-1)}, x_t] + b_o)$	What memory to expose
Hidden State	$h_t = o_t * \tanh(C_t)$	Output / short-term memory

LSTM in Keras

```
from tensorflow.keras.layers import LSTM, Embedding, Dense
from tensorflow.keras import Sequential

# Many-to-One (classification)
model = Sequential([
    Embedding(10000, 128, input_length=200),
    LSTM(64, return_sequences=False),
    Dense(1, activation='sigmoid')
])

# Stacked LSTM
model = Sequential([
```

```

Embedding(10000, 128, input_length=200),
LSTM(64, return_sequences=True), # pass sequences to next LSTM
LSTM(32, return_sequences=False),
Dense(1, activation='sigmoid')
])

```

Lecture 4: GRU Introduction

What is GRU?

GRU (Gated Recurrent Unit, Cho et al. 2014) is a streamlined alternative to LSTM that solves the vanishing gradient problem with fewer parameters and simpler architecture.

- GRU merges cell state and hidden state into one: h_t only.
- GRU uses 2 gates instead of 3: Reset Gate and Update Gate.
- Fewer parameters -> faster training, less memory, better for small datasets.

Design insight

LSTM: we need a memory cell and three gates to control it.

GRU: Forget and Input gates are often complementary.

If forgetting a lot, probably writing a lot -> merge into one Update Gate.

Result: same capability, simpler architecture.

GRU Gates Overview

Gate	Controls Effect
Reset Gate (r_t)	How much of h_{t-1} to use for new candidate. $r_{t-0} \rightarrow$ ignore past. $r_{t-1} \rightarrow$ fully use past.
Update Gate (z_t)	How much of old state to keep vs replace. $z_{t-1} \rightarrow$ keep old. $z_{t-0} \rightarrow$ replace with candidate.

Update Gate = Forget + Input

The Update Gate plays BOTH the LSTM Forget Gate AND Input Gate roles.

This is what makes GRU more compact: one gate does the work of two.

LSTM vs GRU — Architecture Comparison

Feature	LSTM	GRU
States	2 (h_t and C_t)	1 (h_t only)
Gates	3 (Forget, Input, Output)	2 (Reset, Update)
Parameters	More	~25% fewer
Training speed	Slower	~20-30% faster
Long sequences	Slightly better (500+ steps)	Often comparable
Small datasets	Overfitting risk	Better generalisation

Use Case	Choose
Small dataset, limited compute, quick prototype	GRU
Very long sequences, maximum accuracy	LSTM
In practice	Benchmark both — pick on val performance

Lecture 5: Working of GRU

GRU Computation — Full Step

At each time step t , GRU receives x_t (input) and $h_{(t-1)}$ (previous state):

```
# Step 1: Reset Gate
# r_t = sigmoid( W_r * [h_(t-1), x_t] + b_r )
# Controls how much of h_(t-1) enters the candidate computation

# Step 2: Update Gate
# z_t = sigmoid( W_z * [h_(t-1), x_t] + b_z )
# Key gate: balances old state vs new candidate

# Step 3: Candidate Hidden State
# h~_t = tanh( W_h * [r_t * h_(t-1), x_t] + b_h )
# r_t gates how much past info enters the candidate

# Step 4: Final Hidden State
# h_t = (1 - z_t) * h~_t + z_t * h_(t-1)
#
# z_t = 1 -> h_t = h_(t-1)      [old memory fully preserved]
# z_t = 0 -> h_t = h~_t         [fully replaced by candidate]
```

GRU Step Summary Table

Step	Operation	Output	Purpose
1	$r_t = \text{sigmoid}(W_r[h_{(t-1)}, x_t] + b_r)$	Reset (0-1)	How much past to use in candidate
2	$z_t = \text{sigmoid}(W_z[h_{(t-1)}, x_t] + b_z)$	Update (0-1)	Keep old vs write new state
3	$h\sim_t = \text{tanh}(W_h[r_t h_{(t-1)}, x_t] + b_h)$	Candidate	New info to potentially write
4	$h_t = (1 - z_t) h\sim_t + z_t h_{(t-1)}$	New hidden state	Final blended output

How GRU avoids vanishing gradients

When $z_t \sim 1$ for many steps: $h_t = z_t * h_{(t-1)} \sim h_{(t-1)}$
 Gradient flows back multiplied by ~ 1 at each step -> no shrinkage.
 Same principle as LSTM's additive cell state update.

GRU vs LSTM — Gradient Flow

Architecture	How Vanishing Gradient is Avoided
LSTM	Additive cell state: $C_t = f_t C_{(t-1)} + i_t C\sim_t$ -> addition preserves gradient
GRU	Interpolation: $h_t = z_t h_{(t-1)} + (1 - z_t) h\sim_t$ -> when $z_t \sim 1$, gradient passes unchanged

GRU in Keras

```
from tensorflow.keras.layers import GRU, Embedding, Dense, Bidirectional
from tensorflow.keras import Sequential

# Standard GRU
model = Sequential([
    Embedding(10000, 128, input_length=200),
    GRU(64, return_sequences=False),
```

```
Dense(1, activation='sigmoid')
])

# Bidirectional GRU (reads forward + backward)
model = Sequential([
    Embedding(10000, 128, input_length=200),
    Bidirectional(GRU(64)), # 128 total units
    Dense(1, activation='sigmoid')
])

# Stacked GRU
model = Sequential([
    Embedding(10000, 128, input_length=200),
    GRU(64, return_sequences=True),
    GRU(32, return_sequences=False),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Empirical note

GRU and LSTM perform comparably on most tasks.

LSTM has a slight edge on very long sequences (500+ steps).

GRU trains ~20-30% faster and uses less memory.

Best practice: benchmark both on your data and pick on validation performance.

End of Module 21 Notes — Generative AI Course